Who merges what, and when

Updated plain-English report: required ML and Agentic RAG, with the original core dependencies preserved.

What changed: C owns three new ML tasks, including the ML backend and frontend. C05 and C06 are required. **Do not run B05 separately.** A, B and D keep their existing coding assignments; they receive no extra ML coding task.

Who does what?

Person	Main responsibility
A = teammate 1	Website, review screens, premium 3D and release packaging.
B = teammate 2	Duplicate detection, screening rules and the final evaluation report.
C = teammate 3	PICO, LangGraph, gateway/RAG, plus all ML model, backend, frontend and tests.
D = teammate 4	Core backend, database, paper imports, shared services and operations.

The words in the plan

main: the shared accepted code.
Branch: a separate place to build one change.
Pull request (PR): a request to add that change to main.
Reviewer: a teammate who checks the change; they do not take over its coding.
Merge: add the checked change to main.
CI / CI Gate: GitHub's automatic checks. Passing checks are not automatic AI review.

What does "bootstrap" mean?

It means **initial shared setup**: put the existing plan, prompts, locked dependencies and CI files into the team's GitHub repository once. Run the checks and set the review rules. In an empty repository, this can be the first commit; in an existing one, use a reviewed PR.

D01 is the first application coding task. Once D01 merges, A01, B01, C01 and D02 can be built in parallel. No prompt launches another prompt automatically.

This report replaces the earlier PDF's optional ML/RAG wording. It describes planned work, not completed implementation. The original prompt files remain preserved; use the ML addendum for the latest scope.

Build the foundation, then add ML

A prompt ID identifies an assignment, not its position in the whole queue. C-ML01 is the new ML task; it is different from the existing PICO task C01.

Task / builder	What gets added	Must already be merged	Checked by
D01 D builds	Shared backend, database starting point and agreed data formats.	Shared setup	A, B, C
A01 A builds	Website shell, layouts and visual style.	D01	D
B01 B builds	Find and group duplicate papers safely.	D01	D, C
C01 C builds	Extract PICO text: people, intervention, comparison and outcomes.	D01	A, B
D02 D builds	Fetch and save papers from PubMed and Europe PMC.	D01	A
B02 B builds	Screen papers using clear rules and reasons.	B01	D, C
A02 A builds	Search and paper-import screens.	A01, D02	D
C-ML01 C builds	Train and evaluate the ML model; build reusable local inference.	D01, C01, B02	Team approval*
D03 D builds	Save review decisions and connect the basic processing steps.	D02, B02, C01	A, C

* For each new ML task, C implements and tests the work; obtain normal team PR approval. This does not assign ML coding to another developer. Green rows are the new ML tasks.

Why D01 first? Why can others continue?

D01 agrees how records and results are represented. That lets the others build compatible parts. C-ML01 needs those formats, C01 and B02's screening rules, but it stays in a separate ML module.

C-ML01 does not block D03. Its position above is one convenient example. D03 may merge when D02, B02 and C01 are ready, even if C is still training the model. The hard rule is the dependency column, not waiting for every row above.

Connect the working app and ML

These are the unchanged core integration steps, followed by C's new full-stack ML assignment.

Task / builder	What gets added	Must already be merged	Checked by
C02 C builds	LangGraph flow, saved progress and human-review pauses.	C01, D03	D
A03 A builds	Review screen, duplicate controls and PICO editor.	A02, D03	C, D
D04 D builds	Connect website, API and workflow. Enable browser checks.	D03, C02, A03	A, C
B03 B builds	Measure baseline quality and test difficult examples.	B02, D04	C, D
C03 C builds	Check permissions, workflow safety and safe restart behavior.	C02, D04	D, A
A04 A builds	Premium 3D polish, accessibility and usable fallbacks.	A03, D04	D, C
C-ML02 C builds	Connect ML to the backend AND frontend; add real integration tests.	C-ML01, D04, B03, C03, A04	Team approval*

* Normal team PR approval applies to C's ML change. C handles the implementation, tests and any ML connection fixes.

What runs at the same time?

After D03: C02 and A03 can run together once their other prerequisites are met.
After D04: B03, C03 and A04 can run together.
After those checks and C-ML01: C can do C-ML02 while the others continue their existing work.

Why wait until A04 and D04 for ML integration?

C connects ML to a working website and backend, using their merged interfaces. C builds the ML screen and endpoints in separate feature folders, then makes only the small route/navigation connections. A and D do not have to build those ML pieces.

No hidden handoff: C-ML02 must deliver a real working screen calling the real ML backend. A notebook, a mock score or an endpoint with no UI does not finish this task.

Finish RAG, verify ML, then release

C05 and C06 are required under the latest instruction. The Level 2 bonus chain can be inserted before them if selected; see page 5.

Task / builder	What gets added	Must already be merged	Checked by
C05 C builds	Required LLM gateway, limits and validated extraction.	C03, B03, A04	D, B
C06 C builds	Required Agentic RAG using saved papers and verified citations.	C05	D, B
D06 D builds	Backups, secure settings, RAG backend connection and release preparation.	D04, B03, C03, A04, C06	A, C
C-ML03 C builds	Verify ML with release settings; fix integration and deliver measured evidence.	C-ML02, C05, C06, D06	Team approval*
B06 B builds	Final evaluation report, reusing the completed ML evidence supplied by C.	B03, D06, C-ML03	C, D
A06 A builds	Release package, demo, required RAG UI and final browser acceptance.	A04, D06, B06, C03, C-ML03	D, B, C

* Normal team PR approval applies. C handles the new ML work; A/B/D keep their existing release responsibilities.

The exact release handoff

D06 now also waits for C06. D connects the required RAG backend using the integration work already in D06. C owns ML integration independently.
C-ML03 runs after D06. C verifies the model, API, ML screen, saved results and restart behavior against the release settings.
B06 and A06 wait for C-ML03 before final acceptance. They can prepare the report and demo earlier. B reuses C's completed ML results; A receives an already-working ML screen.

One final check: when A06's package is ready, C checks ML against that package before release sign-off and fixes ML-specific packaging issues. This is C's follow-through on C-ML03, not a fourth ML prompt or new ML coding for A.

Required RAG needs C05/C06, D06 backend wiring and A06's working user interface. An unavailable model key or untested live path must be reported honestly; fixture tests alone do not prove live model quality. A rules/manual fallback remains available but does not replace required ML/RAG delivery.

Keep the core order intact

There are 22 required tasks in the updated release path, plus four Level 2 bonus tasks. The old B05 experiment is not selected because C now owns ML.

One valid extended merge queue

Foundation: D01 - A01 - B01 - C01 - D02 - B02 - A02
Insert: C-ML01
Core integration: D03 - C02 - A03 - D04 - B03 - C03 - A04
Insert: C-ML02
Bonus, if selected: D05 - B04 - C04 - A05
Required RAG and operations: C05 - C06 - D06
Insert: C-ML03
Final report and release: B06 - A06

This keeps the relative order of the selected original tasks. It is an example queue, not a rule that all work must be sequential. A task may proceed when its own listed prerequisites are merged. Unrelated original work does not wait merely because an ML task appears earlier in the example.

Optional Level 2 implementation

Task / builder	What gets added	Must already be merged	Checked by
D05 D builds	Saved versions, decision history and exact restore services.	D04, B03, C03, A04	B, C, A
B04 B builds	Tests that grouping changes preserve reviewer decisions.	B03, D05	D, C
C04 C builds	Rerun only changed processing steps safely.	C03, D05, B04	D, B
A05 A builds	History, changes, conflicts and restore screens.	A04, D05, B04, C04	D, B

If you claim the complete Level 2 experience, finish all four tasks before D06 finalizes that feature. If there is not enough time, leave this bonus chain out and still include the required one-page Level 2 design. Skipping it must not remove required ML or RAG.

What B should do about B05

Do not execute B05 separately. Its old file stays in the folder, but its ML role is covered by C-ML01/02/03. B continues B01, B02, B03, optional B04 and B06. This avoids two developers building different ML systems.

What C must actually deliver

C owns the complete ML feature. The model prioritizes papers for human review; it does not automatically overwrite eligibility or reviewer decisions.

C-ML01: train and evaluate a real model

Use the planned scikit-learn TF-IDF and logistic-regression pipeline. Prepare permitted review-specific labels, separate training and testing data, train the model, save a trusted versioned artifact and implement local inference. Record genuine measurements and limitations. A tiny synthetic model is useful for automated tests, not evidence of real-world quality.

Prompt file: prompts/dev-C/07-ml-foundation.md

C-ML02: connect the backend AND frontend

Add authorized ML endpoints and saved results using the existing services. Build a screen that shows ranked papers, model identity, a baseline-order option, errors and model-unavailable states. Register its routes and navigation. Add real backend, frontend and browser tests. No new ML implementation ticket goes to A or D.

Prompt file: prompts/dev-C/08-ml-full-stack-integration.md

C-ML03: prove the release works

Verify startup, a trusted model load, actual inference, screen behavior, restart, performance and decision preservation. Test coexistence with the required gateway/RAG path. Provide the final ML report, commands and demo example. Check the final A06 package before release sign-off.

Prompt file: prompts/dev-C/09-ml-release-verification.md

How to avoid disrupting the other developers

C works mainly in separate ML backend, frontend and test folders. Shared edits are limited to the connections needed to expose the feature. C updates from main first, makes small changes, resolves any ML overlap and reruns tests. If two PRs touch the same connection, merge those PRs one at a time; everyone can still work on unrelated files.

Parallel team work does not mean unlimited capacity for C. Allow roughly 3-5 focused hours for C-ML01, 3-5 for C-ML02 and 2-4 for C-ML03, with more time if usable labels are missing. C also owns PICO, LangGraph and RAG. Prioritize required work over bonus C04; do not silently drop ML/RAG to meet an optimistic schedule.

Read first: prompts/dev-C/ML-ADDENDUM-EXECUTION.md. Exact dependencies come from prompts/tasks.json plus prompts/dev-C/ml-supplement.tasks.json. This PDF explains the combined plan; it does not launch agents or record completed merges.

Who checks, and who clicks Merge?

Suggested arrangement: D coordinates merges, with A as backup. The team can rotate this role; the coordinator is not a fifth developer.

Building, reviewing and merging are separate actions. D01 does not automatically run A01, B01 or C01 as reviews. Ask people or agents to review D01 explicitly. Start their implementation prompts only when the needed earlier work is merged.

Repeat this for every prompt

1. Check prerequisites. Verify the needed tasks are actually in main. A draft PR or a message saying "done" is not enough.

2. Create a task branch from current main. Build one assigned prompt, not a permanent branch containing every task.

3. Implement, test and write the handoff. State what works, what was checked and any remaining blocker.

4. Open a pull request and ask for review. The listed reviewers check their relevant areas. For new ML tasks, C owns coding and tests; normal team approval still applies.

5. Fix review findings and failing checks. A green CI Gate does not replace human review. An agent reviewing its own work is not independent approval.

6. Coordinator merges the approved change. Confirm dependencies, checks and file conflicts are resolved before using the agreed Merge option.

7. Record the merge and update dependent branches. Save the merged commit reference in the handoff. The next developer gets latest main before relying on it.

A concrete D01 example

D implements D01 and opens a PR. You ask A: "Review this D01 PR for frontend data/API needs; do not implement A01 yet." Ask B and C to check their interfaces too. D fixes problems. After review and passing checks, the coordinator merges D01. Then you explicitly assign A01, B01, C01 and D02, which can proceed in parallel.

The final release checklist

Core app: search, import, duplicates, review and PICO work.
ML: trained artifact, real inference, connected UI and C's measured evidence.
Agentic RAG: required C05/C06 plus D06/A06 integration, evidence and clear failure handling.
Team: no duplicate B05 work; C-ML03 accepted before final B06/A06 acceptance; C checks final packaging.
Truthful status: distinguish implemented, tested, live-provider-verified and deployed.

The dependency graph and original task order were checked against the saved manifests. This report describes the current implementation plan; it does not certify model quality, a successful deployment or completed developer tasks.